
Python Review Exercises

Important Instruction for Week 03 Lab:

Do not use AI tools or external code generators to complete these exercises. They are intended for you to practice your own coding abilities. Think through the logic, reference Python syntax, and work through the challenges independently.

✨ These exercises aim to reinforce students' understanding of key Python concepts, including iteration, lists, tuples, dictionaries, and functions. Each section includes a brief description of the topic followed by practice exercises.

 Your task is to Write a Python Programming Language to complete the exercises below:

I. Iteration: for and while, break, and continue

Description: Iteration allows us to execute a block of code multiple times. The for loop iterates over a sequence (like a list or string), while the while loop continues until a specified condition is no longer true. The break statement exits the loop, and continue skips the current iteration and moves to the next.

Exercises:

1. Write a for loop that prints each element in the list `numbers = [1, 2, 3, 4, 5]`.
2. Create a while loop that counts from 1 to 10 and prints the numbers. Use `break` to exit the loop when the count reaches 6.
3. Modify the previous while loop to use `continue` to skip printing the number 4.

II. List: Definition, Methods, Syntax

Description: A list is a mutable sequence that can hold multiple items. It is defined by enclosing items in square brackets, and various methods can manipulate the list (e.g., `append`, `remove`, `sort`).

Exercises:

1. Create a list of your favorite fruits and print it.
2. Add a new fruit to the list using the append method and print the updated list.
3. Remove a fruit from the list using the remove method and print the updated list.
4. Sort the list in alphabetical order and print it.

III. Tuple: Definition, Methods, Syntax

Description: A tuple is an immutable sequence of items. It is defined by enclosing items in parentheses. While tuples do not support item assignment, they can be unpacked or converted into a list for manipulation.

Exercises:

1. Create a tuple containing five integers and print it.
2. Attempt to change one of the integers in the tuple and check your result in the terminal (this should raise an error).
3. Convert the tuple into a list and print the list.

IV. Dictionary: Definition, Methods, Syntax

Description: A dictionary is a mutable collection of key-value pairs. It is defined by enclosing pairs in curly braces. Dictionaries allow efficient access and modification of data based on keys.

Exercises:

1. Create a dictionary to store the ages of three people (e.g., {"Alice": 30, "Bob": 25, "Charlie": 35}) and print it.
2. Add a new person to the dictionary and print the updated dictionary.
3. Remove one person from the dictionary using the pop method and print the updated dictionary.
4. Print all keys and values in the dictionary using a loop.

V. **Function: Return Type and Non-Return Type, Anonymous Function (Lambda Function)**

Description: Functions in Python are defined using the def keyword and can return values. Functions without a return statement implicitly return None. Lambda functions are small anonymous functions defined with the lambda keyword.

Exercises:

1. Write a function named add that takes two numbers as arguments and returns their sum.
Call the function and print the result.
2. Create a function named greet that prints "Hello, World!" without returning a value.
Call the function.
3. Write a lambda function that takes one argument and returns the square of the number.
Use this lambda function to print the square of 5.



(Bonus) User Authentication System

Scenario:

Create a secure user authentication system that includes:

1. **Register:** Users can register by providing a username and a password. The password must be validated for strength (strong or weak).
2. **Login:** Registered users can log in using their username and password.
3. **Forgot Password:** Users can retrieve their password by providing their username.
4. **Password Strength Validation:** Ensure that the password is strong by checking for:
 - a. Minimum length of 8 characters.
 - b. At least one uppercase letter.
 - c. At least one lowercase letter.
 - d. At least one digit.
 - e. At least one special character.
5. **Data Validation:** Prevent users from registering with a username that already exists.
6. **Iteration:** Implement iteration using for, while, break, and continue in the system's logic.
7. **Data Handling:** Use a list to store usernames and a dictionary to store username-password pairs.
8. **Use of Functions:** Implement each feature (Register, Login, Forgot Password) as a separate function.
9. **Lambda Functions:** Use lambda functions where appropriate, such as in password validation or other operations.

Tasks:

1. **Register:**
 - a. Create a function that registers a user.
 - b. Ensure the username doesn't already exist by checking the list of usernames.
 - c. Validate the password for strength based on the criteria above.
 - d. Store the username in a list and the username-password pair in a dictionary.
2. **Login:**
 - a. Create a function to allow users to log in with their username and password.

- b. Validate the credentials against the stored username-password pairs.
 - c. Allow users up to three login attempts before blocking further attempts.
3. **Forgot Password:**
 - a. Create a function to retrieve a password when the user provides a valid username.
 - b. If the username doesn't exist, notify the user.
4. **Menu:**
 - a. Implement a menu that allows users to choose between registering, logging in, or recovering their password.
 - b. Ensure users can exit the program.
5. **Password Strength Validation:**
 - a. Use a function to validate if the password meets the specified strength requirements.
 - b. Provide feedback to users on why a password is considered weak and how to improve it.

Extra Requirements:

- Ensure proper use of **lists** (to store usernames), **dictionaries** (to store username-password pairs), and **functions** (to separate logic).
- Use **iteration** (loops) for retrying actions and managing login attempts.
- Incorporate **lambda functions** for concise operations, such as password checks.

Expected Output:

Scenario 1: Successful Registration and Login

Menu:

1. Register
2. Login
3. Forgot Password
4. Exit

Choose an option (1-4): 1

Enter a username to register: user1

Enter a password: abc

Password too short. Must be at least 8 characters.

Enter a password: abcdefgh

Password must contain at least one uppercase letter.
Enter a password: Abcdefgh
Password must contain at least one digit.
Enter a password: Abcdefg1
Password must contain at least one special character.
Enter a password: Abcdefg1@
Registration successful with a strong password!

Menu:
1. Register
2. Login
3. Forgot Password
4. Exit
Choose an option (1-4): 2
Enter your username: user1
Enter your password: Abcdefg1@
Login successful! Welcome, user1.

Menu:
1. Register
2. Login
3. Forgot Password
4. Exit
Choose an option (1-4): 4
Exiting the program. Goodbye!

Scenario 2: Login Failure with Password Recovery

Menu:
1. Register
2. Login
3. Forgot Password
4. Exit
Choose an option (1-4): 2
Enter your username: user1
Enter your password: wrongpassword
Invalid credentials. You have 2 attempts left.
Enter your username: user1
Enter your password: wrongpassword
Invalid credentials. You have 1 attempt left.
Enter your username: user1
Enter your password: wrongpassword
Invalid credentials. You have 0 attempts left.

Too many failed attempts. Access blocked.

Menu:

1. Register
2. Login
3. Forgot Password
4. Exit

Choose an option (1-4): 3

Enter your username to retrieve your password: user1

Your password is: Abcdefg1@

***Reminder: Do not use AI tools** to generate your solution. Think through how you would logically check each condition and write your code accordingly.*



At the end of this lab, you need to produce one file named:

◆ **Team**

[Group Number]_[Team Number]_W03_Lab.py

Example: G1_T8_W03_Lab.py

◆ **Individual (Absent Student)**

Full_Name_W03_Lab.py

Example: Chan_Dara_W03_Lab.py